

Numerical Analysis Assignment 1 — Report

Name:

Hamza Faisal

Burhanuding Patanwalla

Hussam Zubair

Sarmad Ansari

Mujtaba Zaidi

1. Overview

we basically wrote two scripts to test and experiment with taylor's series and the bisection iteration method. Our goal was to explore edgecases and see a practical implementation of the two algorithms in actual use

2. Design choices

2.1 Error handling (both files)

We only really implemented error handling only when our scripts ran into preset violations or invalid user inputs

We define invalid user inputs as inputs where the function becomes undefined or unexecutable

- The following are types of errors we have in our scripts:
 - A `TypeError` is raised when the objective function f is not callable
 - A `ValueError` is raised when interval boundaries violate $a < b$, numerical parameters are invalid ($\text{tol} \leq 0$ or $\text{max_iter} \leq 0$)
- There are situations where we don't have an error such as:
 - **exhausting `max_iter` without meeting the convergence tolerance does NOT raise an exception**; instead, `bisection()` returns normally with `"converged": False`, the latest midpoint p_n , an explanatory `"reason"`, and the iteration history.

2.2 Relative-error safeguard (bisection)

The standard relative error formula:

$$\text{rel_err} = \frac{|p_n - p_{n-1}|}{|p_n|}$$

Breaks down when $p_n \approx 0$, dividing by a near-zero denominator causes numerical overflow, blowing up the relative error even for tiny brackets. To prevent false non-convergence, the code falls back to absolute error $|p_n - p_{n-1}|$ whenever $|p_n| \leq 10^{-12}$. We chose 10^{-12} because it sits safely above machine precision ($\approx 2 \times 10^{-16}$), though the threshold is admittedly arbitrary.

2.3 How `taylor_series` works

For each $k \in [0, n]$, the loop evaluates $f^{(k)}(x_0)$, divides by $k!$, multiplies by $(x - x_0)^k$, and accumulates the term into $P_n(x)$. The Lagrange remainder is built unevaluated as $\frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)^{n+1}$ using a new symbolic variable ξ .

$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)^{n+1}$$

3. Problem 1 — Taylor series results

(i) e^x about 0, evaluated at $x = 0.5$

n	$P_n(0.5)$	absolute error	relative error
1	1.5000000000000000	0.148721270700128	0.0902040104310499
3	1.6458333333333333	0.00288793736679493	0.00175162255629090
5	1.648697916666667	0.0000233540334615423	0.0000141649373223802

As n grows, error drops rapidly matching the Lagrange bound $R_n(0.5) \leq \frac{e^{0.5}}{(n+1)!} (0.5)^{n+1}$, where the factorial denominator quickly outpaces the power term.

(ii) $\sin(x)$ about 0, $n = 7$

- Terms list: `[0, x, 0, -x**3/6, 0, x**5/120, 0, -x**7/5040]`

Even terms vanish because $\sin(x)$ is an odd function: every even derivative is $\pm \sin(x)$, which evaluates to zero at $x_0 = 0$.

(iii) $\ln(x)$ about 1, $n = 4$

- $P_4(1.2) = 0.182266666666667$
- $\ln(1.2) = 0.182321556793955$

- **Absolute error:** 0.0000548901272879598
- **Relative error:** 0.000301062190632742

The approximation is accurate ($\approx 0.03\%$ relative error) because $x = 1.2$ is near $x_0 = 1$ and well within the convergence interval $(0, 2]$.

(iv) $1/(1-x)$ about 0, $n = 5$

x	P_5(x)	true value	absolute error	relative error
0.9	4.685590000000000	10.000000000000000	5.314410000000000	0.5314410000000000
1.5	20.781250000000000	-2.000000000000000	22.781250000000000	11.390625000000000

Convergence & Error Analysis Across Orders:

n	Absolute Error at x=0.9	Absolute Error at x=1.5
5	5.3144×10^0	2.2781×10^1
10	3.1381×10^0	8.6498×10^1
20	1.0942×10^0	4.9891×10^3
50	5.1538×10^{-3}	9.5670×10^8

- **Radius of convergence:** $R = 1$ on $(-1, 1)$, bounded by the pole at $x = 1$.
- **Inside vs. outside:** $x = 0.9$ is inside ($|0.9| < 1$); $x = 1.5$ is outside ($|1.5| > 1$).
- **Behavior as $n \rightarrow \infty$:** Error vanishes toward zero for $x = 0.9$, while partial sums diverge to $+\infty$ for $x = 1.5$.
- **Boundary convergence:** At $n = 5$, the error at $x = 0.9$ exceeds 53% because the ratio term 0.9^{n+1} decays slowly near the boundary $R = 1$, severely degrading convergence speed.

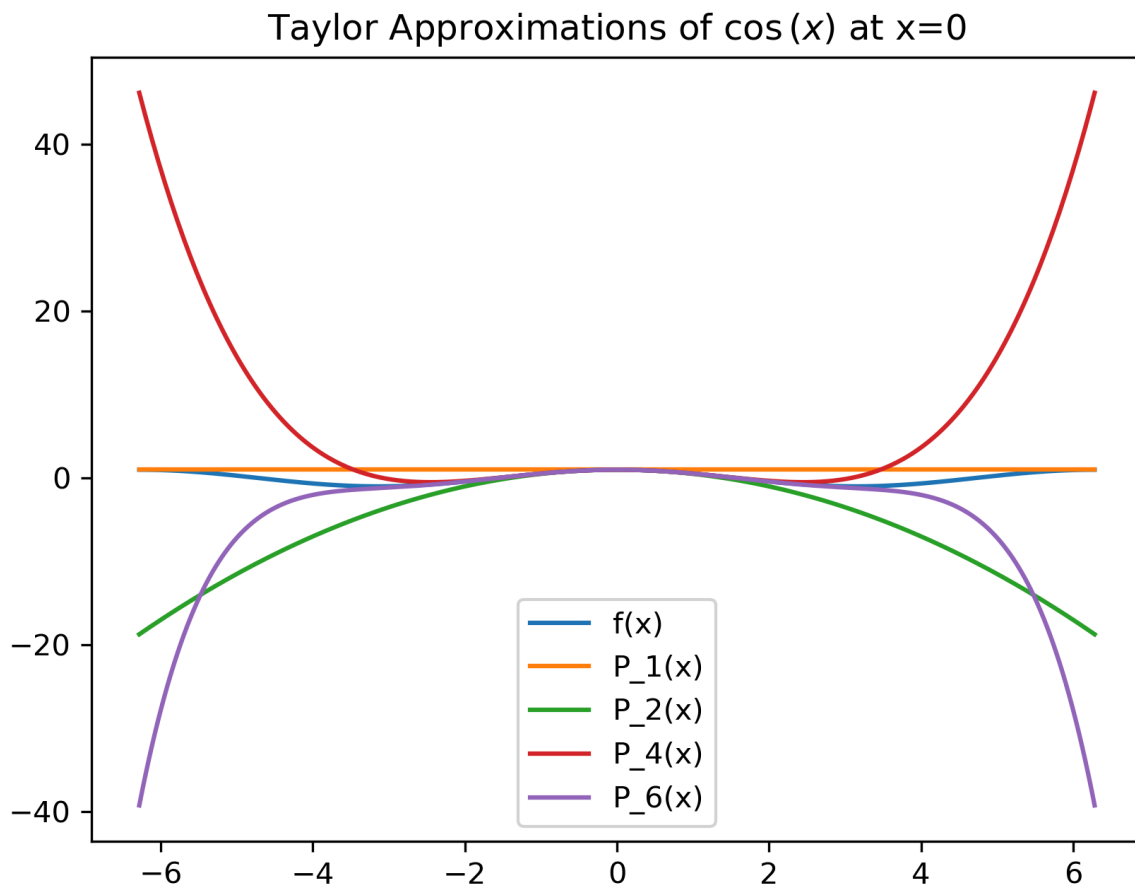
(v) Edge cases

Invalid call	Exception raised	Message
$n = -1$ / non-integer n	ValueError	n must be a positive integer
expr with no free symbols	ValueError	0 or > 1 free symbols
expr with 2 symbols, no var	ValueError	0 or > 1 free symbols

Invalid call	Exception raised	Message
Abs(x) at 0	ValueError	derivative not defined at 0 for n = 1, function is not differentiable at 0
sqrt(x) at 0	ValueError	derivative not defined at 0 for n = 1, limit goes to infinity

For $n=0$, the function returns just the constant term $f(x_0)$ (e.g., 4 for x^2 at $x_0=2$). It handles symbolic expansion points natively, returning $\ln(x_0)+x_0x-x_0$ for $\ln(x)$ around x_0 .

(vi) $\cos(x)$ plot



Each $P_n(x)$ matches $\cos(x)$ near $x_0 = 0$, but diverges further out. Lower orders like P_1 and P_2 fail before $|x| = 2$, whereas P_6 stays accurate past $\pm\pi$. Increasing n widens the domain of convergence where the polynomial aligns with the true wave.

4. Problem 2 — Bisection results

(i) $x^3 - x - 2$ on $[1, 2]$, interval criterion, $\text{tol} = 1\text{e-}6$

- **Root:** 1.5213799476623535
- **Actual Iterations:** 21
- **A Priori Iterations:** 20 ($\lceil \log_2((2 - 1)/10^{-6}) \rceil = 20$)

Why they differ by 1:

$N = \lceil \log_2((b - a)/\text{tol}) \rceil = 20$ measures required halvings. The code checks `interval_w < tol` at the start of the loop *before* halving for that step, requiring one extra check ($n = 21$) to register the halved width below 10^{-6} .

(ii) Comparison of stopping criteria ($\text{tol} = 1\text{e-}6$)

Criterion	Iterations	Root
absolute	20	1.52138042
relative	20	1.52138042
interval	21	1.52137995
residual	22	1.52137971
all	22	1.52137971

- Step vs. Width: $|p_n - p_{n-1}| = w_n/2$, so displacement tests reach tolerance one iteration before interval width.
- Residual: Near the root, $|f(p)| \approx |f'(\alpha)| \cdot |p - \alpha| \approx 5.94 \cdot |p - \alpha|$. The slope amplifies error, making residual stricter and slower.
- "All": Bottlenecks at the slowest criterion (residual).
- A Priori Accuracy: Derived for interval width, it under-predicts when the slope $|f'| > 1$ (20 vs 22 for residual).

(iii) Failure modes

Case	Result	Message / Outcome
$a \geq b$	ValueError	Endpoint 'a' must be strictly less than 'b' ($a < b$). Got $a = 2.0$, $b = 1.0$.
same sign	ValueError	Root is not bracketed: $f(a)$ and $f(b)$ must have opposite signs ($f(a)f(b) < 0$). Got $f(-0.5) = -0.75$ and $f(0.5) = -0.75$.
$f(a) = 0$	Return	Root = 1.0, Iters = 0, Reason = 'Exact root found at left boundary a.'

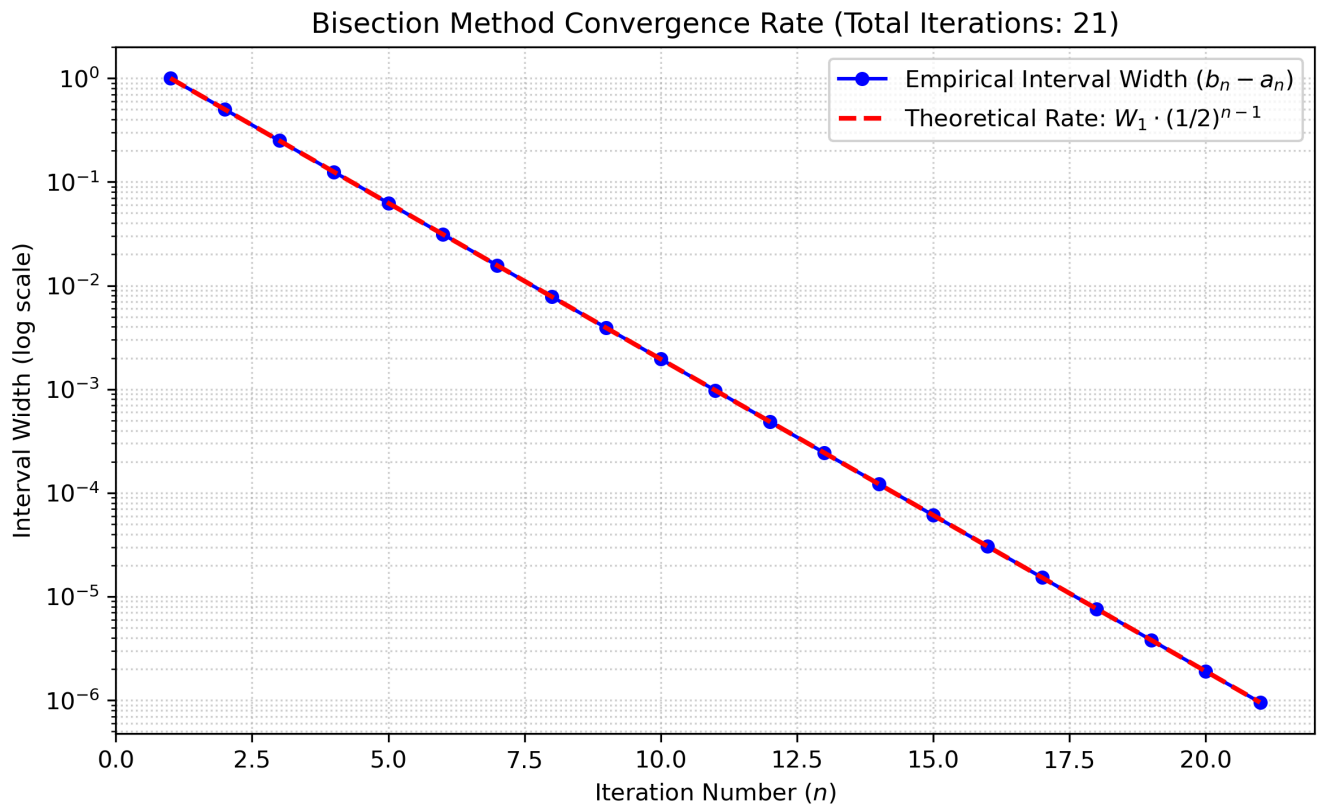
Case	Result	Message / Outcome
$f(b) = 0$	Return	Root = 3.0, Iters = 0, Reason = 'Exact root found at right boundary b.'
$\text{tol} \leq 0$	ValueError	Tolerance has to be greater than 0
$\text{max_iter} \leq 0$	ValueError	Number of iteration cant be 0 or less than 0
bad error_type	ValueError	kindly enter a acceptable error type (absolute, relative, residual, interval, all)
f not callable	TypeError	Parameter 'f' must be a callable function... If you passed a SymPy expression, convert it using sympy.lambdify.
max_iter too low	Return	Converged = False, Reason = 'Maximum iterations (3) reached without satisfying stopping criterion 'absolute'.'
relative error near $p \approx 0$	Return	Root = -9.09e-13, Iters = 40, Converged = True

Design choice: Exhausting iterations is an algorithm status, not invalid usage, so it returns diagnostic state instead of crashing.

(iv) $\sin(4\pi x)$

- **On** $[0, 1]$: $f(0) = 0$ short-circuits at iteration 0, returning 0.0 immediately.
- **Bracket** $[0.1, 0.9]$: Valid because $f(0.1) \approx 0.95 > 0$ and $f(0.9) \approx -0.95 < 0$. Contains roots at 0.25, 0.5, 0.75 and converges to 0.25.
- **Trace**: $p_1 = 0.5$ ($f = 0$, shifts $a \leftarrow 0.5$) $\rightarrow p_2 = 0.3$ ($f < 0$, shifts $b \leftarrow 0.3$) $\rightarrow p_3 = 0.2$ ($f > 0$, shifts $a \leftarrow 0.2$) $\rightarrow$ contracts to 0.25.

(v) Convergence plot



- **Match:** Empirical matches the theoretical $w_1(1/2)^{n-1}$ line exactly.
 - **Slope:** $\log_{10}(0.5) \approx -0.301$ decades/iteration.
 - **Exactness:** Bisection divides the interval in half by construction every step, independent of the function.
-

5. Known limitations

- **Sign-change restriction:** Cannot find roots of even multiplicity where $f(x)$ touches zero without crossing (e.g., $x^2 = 0$).
 - **Single-root capture:** Misses other solutions in multi-root intervals (e.g., $\sin(4\pi x)$), trapping whichever root matches the initial midpoint sign.
 - **Discontinuity failure:** False-converges on vertical asymptotes (e.g., $1/x$ on $[-1, 1]$).
 - **Arbitrary safeguard:** The $\epsilon = 10^{-12}$ fallback for relative error near zero is a heuristic, not mathematically optimal for all scales.
 - **Univariate only:** Taylor series only works for one variable.
 - **Point-only derivative check:** SymPy checks differentiability only at x_0 , ignoring singularities elsewhere in the domain.
-

6. AI use disclosure

TODO: The assignment requires detail on extent and purpose. State plainly what you actually did, e.g. what you asked an AI tool to do (review code against the spec, explain what the report should contain, ...), what it did NOT do, and which suggestions you applied. Be accurate — this section is graded on honesty.